Pemrosesan Bahasa Alami

Deep Learning for NLP From RNN to Transformers

Martin Manullang

Institut Teknologi Sumatera
mctm.web.id

2026

OUTLINE

Peta materi pertemuan ini

1. **Rekap & Jembatan dari Sesi Sebelumnya**

2. **Dasar-Dasar Deep Learning**

Capaian Pembelajaran

Yang bisa kamu lakukan setelah sesi ini

1. Memahami mengapa *Deep Learning* lebih unggul dari ML tradisional untuk teks.
2. Mengenal arsitektur **RNN, LSTM, dan BiLSTM** beserta konsep *attention*.
3. Memahami cara kerja **Transformer** dan *fine-tuning* model pre-trained.
4. Mampu melihat perbandingan performa antar model dari kode yang sudah berjalan.

Fokus Kelas Ini

Penjelasan lebih banyak **menggunakan intuisi visual** daripada rumus matematika yang rumit.

Mengapa Deep Learning?

Keterbatasan yang membuat kita perlu naik level

Pada sesi-sesi sebelumnya kita sudah pakai **TF-IDF + SVM** / **PyCaret**.

Ternyata ada beberapa hal yang **tidak bisa** dilakukan pendekatan itu:

- **Tidak peduli urutan kata** "saya *tidak* suka" vs "saya suka *tidak* ada masalah" dianggap mirip.
- **Tidak paham konteks** kata "bank" di kalimat "tepi sungai" vs "menabung di bank" dianggap sama.
- **Dimensi fitur meledak** setiap kata unik menjadi satu kolom; ribuan kata = ribuan kolom.

Mengapa Deep Learning?

Apa yang ditawarkan Deep Learning

Deep Learning menyelesaikan ketiga masalah tadi dengan cara:

- **Belajar representasi sendiri** model secara otomatis menemukan pola penting dari data, tanpa kita perlu rekayasa fitur manual.
- **Memori urutan** arsitektur RNN / LSTM "membaca" kata satu per satu sambil mengingat apa yang sudah dibaca.
- **Konteks global** mekanisme *attention* memungkinkan model melihat seluruh kalimat sekaligus.

Analogi Sederhana

Kalau TF-IDF seperti **kamus kata** yang hanya menghitung frekuensi, maka DL seperti **orang yang membaca** ia mengerti konteks dan urutan.

Disclaimer Dataset

Peringatan Konten Sensitif

Dataset yang digunakan pada sesi ini berisi teks seputar **kesehatan mental** termasuk topik depresi, kecemasan, dan pikiran bunuh diri.

Dataset ini diambil dari sumber publik dan digunakan **semata-mata untuk tujuan edukasi** penelitian Pemrosesan Bahasa Alami.

Jika kamu merasa terganggu dengan konten ini, silakan sampaikan ke pengajar.

Rekap & Jembatan dari Sesi Sebelumnya

Pipeline Traditional ML

Cara kerja yang sudah kita kenal

Setiap langkah dikerjakan **secara manual** oleh programmer:

- **Fitur dibuat manual**: kita yang memilih cara merepresentasikan teks (TF-IDF, BoW).
- **Model hanya belajar dari fitur tersebut** bukan dari teks aslinya.

Pipeline Deep Learning

Cara kerja yang akan kita pelajari hari ini

Model **belajar sendiri** cara terbaik merepresentasikan teks:

- **Embedding** menggantikan TF-IDF representasi kata *dipelajari* oleh model.
- **Neural Network** sekaligus menjadi ekstraktor fitur *dan* klasifier.

Perbandingan Pipeline: ML vs. DL

Di mana letak perbedaan utamanya?

Traditional ML

- Fitur **dibuat manual** (TF-IDF)
- Tidak peduli **urutan kata**
- Interpretasi lebih **mudah**
- Butuh **data lebih sedikit**
- Training **cepat**

Deep Learning

- Fitur **dipelajari otomatis**
- Memahami **urutan & konteks**
- Model lebih **kompleks**
- Butuh **data lebih banyak**
- Training **lebih lama**

$\Rightarrow$ Tidak ada yang selalu lebih baik pilihan tergantung data, waktu, dan kebutuhan.

Dari Fitur Manual ke Representasi Terpelajar

Lompatan konseptual terbesar dalam Deep Learning

TF-IDF (Fitur Manual)

Setiap kata menjadi satu kolom angka.

Kata	dok.1	dok.2
`sedih`	0.42	0.00
`cemas`	0.00	0.55
`senang`	0.10	0.33

Kata yang mirip maknanya **tidak** saling terkait.

Word Embedding (Terpelajar)

Setiap kata menjadi **vektor kontinu** berdimensi kecil.

Kata	dim-1	dim-2
`sedih`	-0.8	$+0.3$
`cemas`	-0.7	$+0.4$
`senang`	$+0.9$	-0.2

`sedih` dan `cemas` **berdekatan** di ruang vektor karena maknanya mirip.

Dataset: Dari Sesi 1 ke Sesi Ini

Mengapa kita ganti dataset?

Sesi 1 Indonesian Chat

- Bahasa Indonesia
- 2 kelas: *toxic* vs *non-toxic*
- Klasifikasi sederhana
- Cocok untuk ML tradisional

Sesi Ini Mental Health Sentiment

- Bahasa Inggris
- **7 kelas** emosi/kondisi mental
- Nuansa makna sangat halus
- Membutuhkan pemahaman **konteks**

Dataset dengan 7 kelas yang mirip maknanya menjadi **alasan kuat** mengapa kita perlu metode yang lebih canggih dari TF-IDF.

Sekilas Dataset: Mental Health Sentiment

7 Kelas Label yang Akan Kita Prediksi

#	Label	Deskripsi Singkat
1	`Normal`	Teks tanpa indikasi kondisi mental tertentu
2	`Depression`	Ekspresi rasa sedih, putus asa, tidak berdaya
3	`Suicidal`	Pikiran atau niatan menyakiti diri
4	`Anxiety`	Kekhawatiran berlebih, rasa takut, panik
5	`Bipolar`	Perubahan suasana hati ekstrem (senang↔sedih)
6	`Stress`	Tekanan hidup, beban pikiran
7	`Personality Disorder`	Pola pikir atau perilaku yang tidak biasa

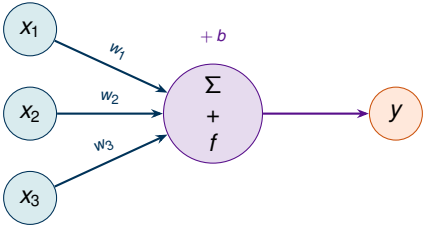
Tantangan: Banyak kelas yang sulit dibedakan misalnya `Depression` vs `Suicidal`. Inilah yang membuat tugasnya menarik!

Dasar-Dasar Deep Learning

Neuron: Unit Terkecil Deep Learning

Meniru cara kerja sel saraf di otak

Bayangkan neuron seperti **saklar cerdas**: ia menerima beberapa sinyal, menimbang-nimbangnya, lalu memutuskan apakah harus "menyala" atau tidak.



- x_1, x_2, x_3 = **input**, w_1, w_2, w_3 = **weight** (bobot kepentingan)
- b = **bias** (pergeseran ambang), f = **fungsi aktivasi**
- Rumus: $y = f\left(w_1 x_1 + w_2 x_2 + w_3 x_3 + b\right)$

Weight & Bias: Parameter yang Dipelajari

Model belajar = model menyetel angka-angka ini

Weight (w)

- Menentukan **seberapa besar pengaruh** suatu input
- w besar $\rightarrow$ sangat berpengaruh
- w negatif $\rightarrow$ bersifat "menghambat"

Bias (b)

- Menggeser titik keputusan neuron
- Neuron bisa aktif meski semua input = 0
- Seperti b dalam $y = mx + b$

Inisialisasi Weight

Dimulai dari nilai acak kecil. Metode umum: **Xavier** atau **He initialization**. Nilai disesuaikan bertahap selama training.

Fungsi Aktivasi

Memberi kemampuan "berpikir non-linear" pada neuron

Tanpa fungsi aktivasi, seluruh jaringan hanya melakukan operasi linear.

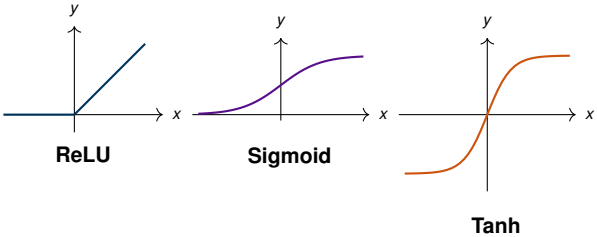
Fungsi	Rentang Output	Kapan digunakan
ReLU	$[0, \infty)$	Hidden layer paling umum, komputasi cepat
Sigmoid	$(0, 1)$	Output layer klasifikasi *biner*
Tanh	$(-1, 1)$	Hidden layer RNN / LSTM mengatasi batasan Sigmoid
Softmax	$(0, 1)$, jumlah = 1	Output layer klasifikasi *multikelas*

Analogi ReLU

ReLU ($\max(0, x)$) seperti lampu yang hanya menyala saat ada listrik positif nilai negatif $\rightarrow$ tetap 0.

Grafik Fungsi Aktivasi

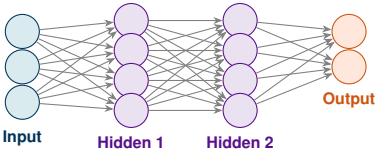
Visualisasi singkat bentuk masing-masing fungsi



Softmax tidak digambar di sini karena ia bekerja pada *vektor* (bukan satu angka) mengubah sekumpulan skor menjadi distribusi probabilitas yang totalnya = 1.

Layer: Menyusun Neuron Menjadi Jaringan

Konsep *depth* dalam Deep Learning



- **Input layer**: menerima data mentah (teks yang sudah di-encode)
- **Hidden layer**: mengekstrak pola makin banyak layer, makin *dalam*
- **Output layer**: menghasilkan prediksi (7 kelas $\rightarrow$ 7 neuron output)

Loss Function: Mengukur Seberapa Salah Model

Sinyal yang memberi tahu model "kamu masih keliru"

Loss function mengukur selisih prediksi model vs. jawaban benar. Makin kecil *loss*, makin bagus.

Cross-Entropy Loss Klasifikasi Multikelas

$$\mathcal{L} = -\sum_{c=1}^{C} y_c \cdot \log(\hat{p}_c)$$

- $y_c = 1$ jika kelas c benar, 0 jika tidak
- $\hat{p}_c$ = probabilitas prediksi untuk kelas c (output Softmax)

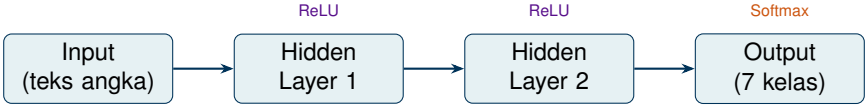
Contoh

Jawaban: `Anxiety`. Prediksi `Anxiety` 90% $\rightarrow$ loss kecil. Prediksi `Normal` 90% $\rightarrow$ loss besar.

Feedforward Network (MLP)

Neural network paling sederhana data mengalir satu arah

MLP (*Multi-Layer Perceptron*) adalah jaringan saraf di mana data bergerak **hanya dari kiri ke kanan** tidak ada putaran balik.



Kelebihan MLP:

- Sederhana dan cepat dilatih
- Cocok untuk data tabular

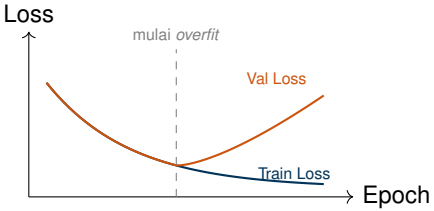
Kekurangan untuk Teks:

- Tidak mempertimbangkan **urutan kata**
- Inilah yang nanti diselesaikan RNN

Overfitting: Model Terlalu "Hafal"

Masalah klasik yang sering muncul saat melatih model

Overfitting terjadi ketika model terlalu fokus menghafal data latih, sehingga performa di data baru (data uji) justru buruk.



Saat *val loss* mulai naik sementara *train loss* terus turun, itulah tanda overfitting.

Regularisasi: Cara Mencegah Overfitting

Tiga teknik yang paling sering digunakan

Dropout

Saat training, sebagian neuron **dimatikan secara acak** di setiap iterasi.

Memaksa model tidak bergantung pada neuron tertentu saja.

Contoh: `Dropout(0.3)` $\rightarrow$ 30% neuron dimatikan tiap langkah.

L2 Regularization

Menambahkan **penalti** pada nilai weight yang terlalu besar ke dalam loss function.

Weight besar $\rightarrow$ model mengandalkan fitur tertentu secara berlebihan.

Disebut juga *weight decay*.

Early Stopping

Hentikan training **saat val loss mulai naik**, bukan saat mencapai epoch maksimum.

Parameter: `patience=3` $\rightarrow$ training berhenti jika val loss tidak membaik selama 3 epoch berturut-turut.